

THESIS FOR BACHELOR'S DEGREE

Self-Adaptive Mutation Operators for Genetic Neural Networks in Survival Analysis

Jonatan Åstrand-Ferris

Department of Astronomy and Theoretical Physics

Supervised by Patrik Edén



LUND
UNIVERSITY

Contents

1	Introduction	4
1.1	Neural networks and genetic algorithms	4
1.2	Genetic algorithm hyperparameters	5
2	Method	5
2.1	Self-adaptive mutation operator	5
2.1.1	Self-adaptive mutation width	5
2.1.2	Mutation operators	5
2.1.3	Extending self-adaptation to other mutation HPs	6
2.2	Adaptive mutation operator	6
2.3	Data sets	6
2.4	Tests of fitness	6
3	Results	7
3.1	C-index validation results	7
3.2	Evolution of features	7
3.2.1	SAGA-1 mutation rates	7
3.2.2	SAGA-1 increasing mutation width	7
3.2.3	SAGA-2 mutation dynamics	7
3.2.4	SAGA-3 operator probability dynamics	8
4	Discussion	8
5	Conclusion	8
6	Appendix	9
6.1	Tables and figures	9

Acknowledgements

I would like to sincerely thank my supervisor Patrik Edén for his guidance and support through this project. Thank you for taking the time with me and for sharing so much of your knowledge. It's been a pleasure to do this project with you.

I would also like to thank Ludwig Siwe for taking the time to explain his work in detail and for always being available to discuss Java programming.

Lastly, I'd like to thank my wife Georgia, without whom it would have been impossible to complete this work, for always supporting me.

Popular science description

How to train your AI: repiton, repeton, repetition

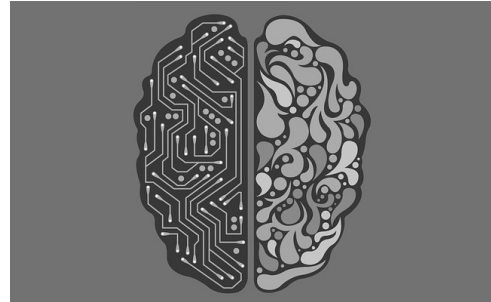
Artificial intelligence(AI) has been on the rise within the medical field worldwide in the last five years. From image recognition software to find skin cancer, to big data analysis creating treatment suggestions, these machines are becoming more common and promising day by day. In the short term, medical professionals will be able to request an AI analysis as easily as they would a blood test. In the long term, we could see machine diagnostics dominating entire medical fields.

As AI becomes accepted throughout society, the shortcomings become more apparent. The most central problem is the lack of general artificial intelligence. There are single-problem oriented programs that do one thing well but fails when applied to other problems. No need to fear an AI take-over any time soon then. Instead, we are definitely at a point in time were human and computer intelligence can join forces and achieve great things that was previously not possible.

This study used machine learning combined with self-adaptive evolutionary training. To understand these concepts, imagine you try and score a three pointer in basketball. You shoot, you miss, you try again. Every time you shoot, you correct your shot, shooting harder or softer, and thus improve the longer you keep going. A machine learns similar tasks in the same way, by repeating and learning from its mistakes. But learning is more complicated than that. You don't just aim higher or lower, you might jump to get a better angle, or change how you move your hands. The same goes for the machine. Each time it tries to solve a problem, it evolves and gets better, changing depending on what change has the biggest positive impact. So how do you know how much to change and when? The machine must be allowed to learn and change in every way it needs for it to solve the problem at hand.

By applying these principles, this work presents an attempt to develop a program that could solve a set of similar but varied medical problems, resulting in SAGA, or Self-Adaptive Genetic Algorithm. The program is different from previous similar programs in that it changes depending on the problem, and it does so without a human intervention. SAGA managed to find solutions that were better than previous static programs, relying less on the user of the program, as well as the data it was given.

The learning techniques that SAGA explored should be of interest to the medical field, as it makes the use of machine learning more accessible, but also to anyone interested in the pursuit of a truly general and independent AI.



Machine and human intelligence combined

Abstract

Artificial neural networks (ANN) are used within the medical field of survival analysis to rank patients according to their risk group. To evaluate how well the ranking was conducted, it is common to obtain the concordance index error (c-index). It has been shown that ANNs can be trained directly on the c-index with the use of genetic algorithms (GA).

The GA evolution of an ANN is controlled by a set of operators, which in turn are governed by hyperparameters. These hyperparameters are usually static and set to generally accepted good values, or optimised through a grid search for each specific data set.

In this article, adaptive and self-adaptive techniques are introduced to the hyperparameters that governs the mutation operators. It is shown that while training on the c-index, it is possible to simultaneously revise the mutation width. Furthermore, it is found that these techniques can be generalised to other aspects of mutation, and show promising results of the possibility of a self-adaptive genetic algorithm, independent of initialisation and data set.

1 Introduction

Artificial neural networks (ANN) and genetic algorithms (GA) are two schematics of learning and optimisation within the discipline of machine learning. ANNs are models inspired by the development of a brains synapses, while GA models are based on biological evolution. In their own right, they posses a range of strengths and weaknesses but they have been shown to be combinational by Davis and Montana [1]. The result was the development of a training algorithm that was empirically shown to be more suitable than previous existing techniques when training on certain complex non-linear problems.

Such a problem exists within survival analysis, namely, the problem of sorting patients into low-risk and high-risk groups as a basis for treatment decisions. In such a problem, patient parameters may very well have non-linear dependencies, which lends itself to be modelled through ANNs.

The problem was solved by Kalderstam et al [2] through training directly on the concordance index(c-index). The c-index compares the model outputs and event times to measure if the outputs are ordered correctly [3]. The use of GAs on ANNs were vital to the success of the approach since the rank based c-index does not allow for gradient descent methods.

Siwe [4] investigated the features of the GA in more detail by studying the hyperparamters (HP), which is the set of parameters governing GA training, and how to optimise them. The study found that training results can be improved by altering HPs during training and that optimal GA training may depend on HP values.

Based mainly on these two previous works, this study aimed to develop a genetic neural network that optimises its HPs throughout training on the c-index. By doing so, the resulting network should rely less on the initialisation of the HPs, resulting in a more general algorithm.

The approach chosen was to incorporate hereditary HPs in the theoretical framework and to introduce adaptive and self-adaptive techniques. In

this case, hereditary HPs refers to local and individual HPs specific to the node of the ANN. If the HPs are updated through a specified intervention when a certain criteria is met, the algorithm is referred to as adaptive. In contrast, if the HPs are continuously updated without specific set rules, it is referred to as self-adaptative [6–8].

This study can be divided into four parts. Firstly, a range of self-adaptive mutation operators are introduced and compared by tests on four medical data sets. These are well-established mutation operators that were tested on a range of different problems previously, as well as some more experimental operators that have additional interesting features. Secondly, the best performing combinations of self-adaption techniques are built into three different stages of mutation operation. The three operators have three levels of self-adaptation and their features are compared in search of interesting behaviour. Thirdly, adaptation is introduced to solve the problems that emerged during training. Finally, the results are evaluated by comparing fitness to simple vanilla GA.

1.1 Neural networks and genetic algorithms

A three-layer multilayer perceptron (MLP) was used, consisting of the input layer, with the same amount of nodes as the features of the data sets studied; a hidden layer, made up of two nodes with tanh activation functions; and an output layer, made up of a linearly activated node.

In this study, and in many general cases, evolution in genetic algorithms is conducted as follows. A population of randomly chosen individuals are initiated, and a maximum number of generations is fixed. For every generation a successive population is generated by the means of the operators *crossover*, *mutation* and *selection*.

Crossover is the process of splitting two individuals into their different components, allowing the components to mix, thus creating a new individual with a different set of components.

Mutation duplicates the individual, but has changed the components of the duplicate, so as to introduce new material into the population. The amount of change introduced is usually in the same order of magnitude as in initiation.

Selection describes the scheme of picking the best performing individuals of each reproduced population, generation by generation. One way is to order the populations from best to worst performance and eliminate the worst, reducing the total population back to its original size. [9]

1.2 Genetic algorithm hyperparameters

In this study, the evolution of each generation in GA is controlled by the previously mentioned operators, which are governed by a set of HPs.

- Initialisation width, γ . Determines the distribution width of the initial weights in the nodes of the MLP.
- Mutation width, σ . Governs the distribution width of the weight change that occurs when a mutation operator is applied.
- Node selection, m . Describes how many nodes ($m \neq 0$) in the MLP that will undergo mutation when a mutation operator is applied.
- Operator probability, o . Decides the probability of applying the mutation operator, and as such, $1 - o$ gives the probability of crossover.
- Parent scalar, p . Controls the probability of picking parents with high fitness. The individual with rank r in fitness is picked with a probability proportional to p^r .

2 Method

2.1 Self-adaptive mutation operator

In section 1.1, *mutation* is described as introducing a random change to a weight, where the change is of the same order of magnitude as initialisation. Since all the weights are initialised from the same distribution, all weights will also be mutated in a similar way.

Self-adaptive mutation gives each node an individual development of its hereditary mutation width. With this in mind, each node, j , can be represented as

$$(\bar{\omega}_j, \sigma_j) \quad (1)$$

where ω_{ij} is the ij th weight, and σ_j is the mutation width of the node. As such, mutation width and weights are usually updated according to

$$\sigma'_j = \sigma_j D_1(\tau) \quad (2)$$

$$\omega'_{ij} = \omega_{ij} + D_2(\sigma'_j) \quad (3)$$

where D_1 and D_2 are distributions with mean zero and standard deviation τ and σ'_j respectively. The constant τ is commonly referred to as the learning rate, and depends on the distribution. It is quite common to set $D_1 = D_2$ [6, 7], but is not the case in this study.

2.1.1 Self-adaptive mutation width

Two methods of self-adaptive mutation width σ were tested.

- Gaussian: $\sigma' = \sigma + \sigma N(0, 1)$
- Exponential: $\sigma' = \sigma \cdot \exp(\tau N(0, 1))$

where $N(0, 1)$ is a number sampled from a normal distribution with mean zero and standard deviation one, using Box-Mueller transformations [10]. The learning rate, τ is dependent on the amount of weights, n , and is scaled according to $\tau \propto \frac{1}{2\sqrt{n}}$. For negative mutation widths, σ was set to a small but non-zero ϵ .

2.1.2 Mutation operators

Self-adaptation was combined and tested with a range of mutation operators. They were all based on sampling numbers from different distributions, D_2 , and combining them with the mutation width. The numbers introduced are $E(0, 1)$, which is obtained through inverse transform sampling from the distribution $f(E) \propto \exp(-|E|)$, and $C(0, 1)$, which is obtained from a Cauchy distribution, also through inverse transform sampling [11].

The four different mutation operators were Exponential (EMO), Gaussian (GMO), Mean (MMO) and Adaptive Mean (AMMO) [8].

- EMO: $\omega'_i = \omega_i + \sigma' E(0, 1)$
- GMO: $\omega'_i = \omega_i + \sigma' N(0, 1)$
- MMO: $\omega'_i = \omega_i + \frac{1}{2} \sigma' (N(0, 1) + C(0, 1))$
- AMMO: $\omega'_i = \omega_i + \sigma'_1 N(0, 1) + C(0, 1) \sigma'_2$

Here, notation of index j for node j has been suppressed.

In this study, there is a significant difference between EMO, GMO and MMO, AMMO. The latter two undergo both crossover and mutation in each generation, and as such do not utilise the operator probability HP, o . This is in accordance

with the original implementation of the operator by Chellapilla and Fogel [8].

Furthermore, AMMO is the only one of the four that utilises a second mutation width, σ_2 .

2.1.3 Extending self-adaptation to other mutation HPs

By utilising the same self-adaption techniques as in section 2.1, it is possible to adapt other HPs as well. This was tested on the HP o , operator probability.

$$o' = \min\{o \cdot \exp(\tau N(0, 1)), \epsilon_1\}$$

with $\epsilon_1 \lesssim 1$ as well as τ and N as described in section 2.1.1

The number of nodes was also self-adapted. However, none of the techniques described so far is fit to explore the integer combinations of node selection. As such, a simple algorithm was conceived.

Algorithm 1 Self-adaption of m

1. Generate a random uniform number between 0 and 1, call it y
 2. if $y > 0.5 \rightarrow$ return m
 3. if $y < 0.25$
 - (a) if m is equal to the maximum amount of nodes in the MLP $\rightarrow$ return $m - 1$
 - (b) else $\rightarrow$ return $m + 1$
 4. else
 - (a) if m is equal to the minimum amount of nodes in the MLP $\rightarrow$ return $m + 1$
 - (b) else $\rightarrow$ return $m - 1$
-

The introduction of three new self-adaptive HPs demanded a new representation of the nodes of the MLPs, as was previously described in eq 1

$$(\bar{\omega}_j, \sigma_{1,j}, \sigma_{2,j}, m, o).$$

2.2 Adaptive mutation operator

Adaptation is fundamentally different from self-adaptation. It is a controlled measure that takes place at a given time, or when a criteria is met, or both. This is in contrast to self-adaptation, where such control measures do not exist.

In this study, adaptation was tested on the mutation width. The intervention was based on the survival rate of mutated offspring, s_m , and is as such implemented as the last step in each generation. If the survival rate was deemed high, the

mutation width increased, and if it was low, the width shrunk. Conditions were set so as to avoid runaway mutation widths. The rate at which survival was deemed high or low was 30 and 5 percent respectively.

$$\sigma' = \begin{cases} 2\sigma, & \text{if } s_m > 0.3 \text{ and } \sigma < 3 \\ 0.5\sigma, & \text{if } s_m < 0.05 \text{ and } \sigma > 0.001 \end{cases} \quad (4)$$

These adaptive techniques were combined with the mutation operators in section 2.1.2

2.3 Data sets

Four different medical data sets were used for testing.

- *PBC(MAYO)*, [12], concerning *primary biliary cholangitis*, where death was the studied event. The data set had 17 input features and contained 312 patients.
- *LUNG*, [13], concerning *Lung cancer*, where death was the studied event. This data set had 7 input features and contained 228 patients.
- *FLCHAIN*, [14], concerning *Free light chains*, where death was the studied event. 7 input features and 314 patients.
- *NWTCTO*, [15], concerning *Wilm's tumor*, where relapse was the studied event. 4 input features and 309 patients.

Each input was normalised to zero mean and unit variance. Missing values were replaced with the mean of obtained values and the data sets were reduced as in Siwe [4].

2.4 Tests of fitness

Validation was obtained by using three-fold cross validation. The training data was split into three equal pieces, where one was used for validation and the remaining for training. This was repeated three times to rotate through all data. An average of ten rounds of cross-validation was obtained, and presented as the validation result.

Gaussian and exponential self-adaptation was combined with the mutation operators. For EMO and GMO, the starting values of the HPs were set to $\gamma = \sigma_1 = 1$, $m = 2$, $o = 0.5$ and $p = 0.9$. MMO and AMMO were initialised in the same way, with the two differences being an additional mutation width $\sigma_2 = 1$ and the lack of an operator probability.

The best performing combination of self-adaptation and mutation operation was then chosen for continued testing, where self-adaptation was applied to operator probability o and node selection, m .

These tests resulted in three main self-adaptive genetic algorithms (SAGA), as well as one adaptive genetic algorithm (AGA):

- SAGA-1, where the mutation width, σ , is self-adaptive.
- SAGA-2, where σ and the number of nodes mutated in each generation, m , are self-adaptive.
- SAGA-3, where σ , m and the probability of mutation, o , are all self-adaptive.
- AGA, where σ is adaptive.

To evaluate SAGA-1,-2,-3 and AGA, several tests were done where the mutation width was varied from 0.1 to 2.5, with 5 iterations per value. The results were compared to tests of a solely hereditary GA, which lacks the adaption and self-adaption described in section 2.1.1. However, since the mutation width is initialised from a distribution depending on initial σ , the evolution of the HP is determined through elimination of individuals. The results were also compared to that of vanilla GA, where all the individuals of the population had the same HP values through the entire training process.

Finally, L2-regularisation [16] was introduced to address the signs of overtraining that was observed during tests.

3 Results

3.1 C-index validation results

It was found that Gaussian self-adaptation together with EMO performed the best on average over all data sets. Hence, it was chosen for further testing. The complete results of the tests can be viewed in Table 1.

For the main test, every value of σ was iterated five times. It was found that SAGA-1,2 and 3 performs better than vanilla GA on FLCHAIN in this particular test. Furthermore, AGA indicated a slight edge when training on NWTCO. However, when training on PBC and LUNG there is no significant improvement of validation with either SAGA or AGA. All these results can be found in Table 2.

To study the dependencies on initial HP values, comparisons of the ten ensembles with highest validation results were made focusing on the initial mutation width. It was found that initial mutation width averages among the best performing populations were higher with SAGA and AGA, meaning that a wider range of initial values resulted in equally good solutions compared

to vanilla GA. These results can be found in Table 3.

Further testing using L2-regularisation was conducted. However, no improvement on validation results was found on any of the four data sets.

3.2 Evolution of features

The presented results are for those cases where relevant properties for the different genetic algorithms are shown clearly. All populations were based on the same HP initialisation as previous tests if nothing else is specified.

3.2.1 SAGA-1 mutation rates

SAGA-1 populations were tested with low probability of undergoing mutation ($o = 0.1$) on the NWTCO dataset. As can be seen in figure 1a, the mutation width still decreased even though mutation survival was close to zero. Final validation fitness was 0.979.

When training another set of SAGA-1 populations, initialised with high probability of mutation ($o = 0.9$), it could be observed that the mutation width was adjusted at a similar rate, see Figure 1b, with final validation fitness at 0.984.

Interestingly, self-adaptation of mutation width did not seem to be dependent on the mutation rate. Furthermore, the survival rate of both mutation and crossover is shown to benefit from a higher mutation rate.

3.2.2 SAGA-1 increasing mutation width

Setting the initial mutation width, σ_1 , close to zero, self-adaptation seem to be able to quickly increase mutation widths. An example of this is shown in figure 1c with SAGA-1 on LUNG. Even at such a sub-optimal initialisation, validation fitness was able to reach 0.709, which is very close to the average performance within 250 generations on this data set.

3.2.3 SAGA-2 mutation dynamics

SAGA-2 evolves its mutation differently to SAGA-1 in the sense that it is able to rapidly decrease or increase mutation in the MLP by allowing more or less nodes to mutate. This creates a dynamic where mutation of a weight might not happen as often, but when it does it is wilder. Also observed is the dynamic where all the nodes are allowed to mutate the weights, but only a little. A comparison between SAGA-1 and SAGA-2 on PBC, obtaining the same validation result, can be seen in Figure 1d. SAGA-2 seems to prefer a rapid decrease in the average number of nodes, resulting in a mutation width that stays wider for longer.

3.2.4 SAGA-3 operator probability dynamics

SAGA-3 introduces adaptation of probability of mutation, the HP o , on top of the framework of SAGA-2. This gives SAGA-3 complete control over all the HPs that governs the mutation operator. With the initiation of $o = 0.5$, there is barely any development of the HP, which can be seen in Figure 2. It should be possible to decrease or increase the operator probability during training if there is a preference for either mutation or crossover. However, this is not observed in this study.

4 Discussion

When comparing vanilla GA to hereditary GA, SAGA and AGA, there is some indication that the validation result can be improved, or at least be equivalent, by the use of adaptive or self-adaptive hereditary HPs when training on these particular data sets.

The improvement from vanilla to self-adaptive GA can be explained by how the individual mutation width affects crossover and mutation. Without an individual mutation width, every node that goes through a round of mutation will do so with the same mutation width. If the mutation width is "bad", e.g. too large or too small, every node will mutate either wildly, or not enough, resulting in a poor training result overall for the MLPs and as such they would not survive the generation. If instead there are individual mutation widths, some might perform well and others might not. The ill-performing MLPs will to a larger degree be killed off, following the same line of argument as before. The others will to a larger degree survive and will have a chance to breed in the next generation, spreading the "good" mutation width to new MLPs through crossover. This results in "good" mutation widths throughout the entire population.

Tests of SAGAs on FLCHAIN display a significant improvement of validation results, with a slightly higher c-index on the average result, see Table 2. This increase can be interpreted as an improvement on the worst fitness results, as the worst measured fitness was increased by approximately 0.01. No significant improvement of the validation result can be observed for PBC, NWTCO and LUNG. However, a equivalent result is reached. SAGA and AGA can not improve the validation result if the task is solved optimally by a simpler algorithm. This could be the case for PBC and NWTCO, since training on PBC

seem to prefer a close-to-zero mutation width and on NWTCO mutation width seem to be of little importance, see Table 3. It is also well documented that simple self-adaptive techniques such as SAGA-1 have a tendency to become stuck in local minima [6], which might explain the limited results of the LUNG data set.

SAGA-2 and SAGA-3 are experimental algorithms, and they introduce more flexibility in the search of the solution space. SAGA-2 nodes are updated in a step-wise manner, with only one step possible at each given time. This is of course problematic for larger MLPs as the efficiency of the algorithm is decreased proportionally to the size of the MLP. As such, it should be modified to include larger steps if to be applied to a larger MLP. SAGA-3 operator probability was shown to prefer values of close to equal probability between crossover and mutation for nearly all tests and data.

A similar technique to those used in this study is CMA-ES, "Covariance Matrix Adaption Evolutionary Strategy", developed by Hansen et al in 1995 [17]. A covariance matrix is used to increase the likelihood of a successful weight-change, $\Delta\bar{w}_j$, based on weight-changes of surviving offspring in previous generations. As with GAs generally, CMA-ES is gradient-free by design which is suitable when using the c-index as error function.

Another approach is to investigate more complex self-adaptation schemes, for example, NEAT, "Neuroevolution of Augmented Topologies", where the whole architecture of the MLP is self-adapting throughout training starting with the smallest possible MLP [18]. This might be fruitful, especially when dealing with FLCHAIN and LUNG, as the limited results obtained in this study, as well as previous studies, could indicate a potential optimisation possibility in the design of the ANN architecture.

5 Conclusion

It has been shown that it is possible to adjust hyperparameters concurrently as weights are trained on the c-index fitness function. This adjustment can be conducted by adapting or self-adapting techniques. With the mutation operator adopted by SAGA and AGA, mutation can be sufficiently adapted to keep it relevant for training through most of the generational span. Furthermore, this study shows that it seems reasonable to suggest that a self-adaptive or adaptive genetic algorithm can obtain good solutions and sometimes even improve validation results when compared to ordinary GA.

6 Appendix

6.1 Tables and figures

Table 1: Gaussian and EMO was found to produce the best, or numerically close to the best, average validation results and was used in subsequent tests.

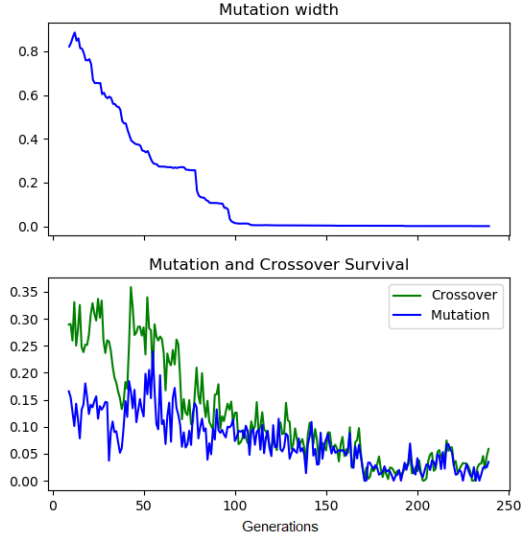
Self-adaptative mutation width distribution	Mutation operator	Data	Fitness(C- index)
Gaussian	EMO	NWTCO	0.986
Exponential	EMO	NWTCO	0.985
Gaussian	MMO	NWTCO	0.968
Exponential	AMMO	NWTCO	0.966
Expo	GMO	NWTCO	0.964
Gaussian	GMO	NWTCO	0.964
Expo	MMO	NWTCO	0.961
Gaussian	AMMO	NWTCO	0.961
Exponential	EMO	PBC	0.894
Gaussian	EMO	PBC	0.892
Exponential	MMO	PBC	0.879
Gaussian	AMMO	PBC	0.878
Gaussian	MMO	PBC	0.875
Exponential	GAUSSIAN	PBC	0.874
Exponential	AMMO	PBC	0.874
Gaussian	GMO	PBC	0.873
Exponential	GMO	FLCHAIN	0.772
Exponential	MMO	FLCHAIN	0.772
Exponential	AMMO	FLCHAIN	0.771
Gaussian	EMO	FLCHAIN	0.771
Gaussian	GMO	FLCHAIN	0.771
Gaussian	AMMO	FLCHAIN	0.77
Gaussian	MMO	FLCHAIN	0.764
Exponential	EMO	FLCHAIN	0.758
Gaussian	EMO	LUNG	0.71
Gaussian	MMO	LUNG	0.706
Exponential	EMO	LUNG	0.704
Exponential	AMMO	LUNG	0.702
Exponential	GMO	LUNG	0.7
Exponential	MMO	LUNG	0.695
Gaussian	AMMO	LUNG	0.695
Gaussian	GMO	LUNG	0.693

Table 2: Average validation results for the different models, $\sigma \in [0.1, 2.5]$. Significant improvement was found on validation results for FLCHAIN using self-adaptive models.

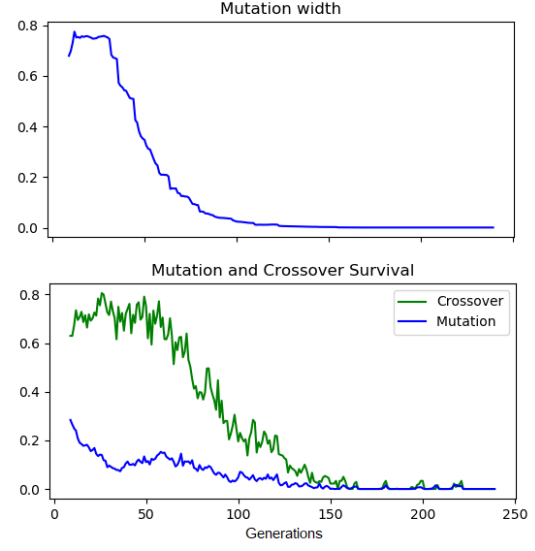
DATA	RESULT	VANILLA	HEREDITY	SAGA-1	SAGA-2	SAGA-3	AGA-1
PBC	Ave. fit	0.887	0.889	0.891	0.891	0.890	0.891
	Std. dev	0.007	0.003	0.003	0.002	0.003	0.003
	Best fit	0.902	0.895	0.896	0.896	0.895	0.897
	Worst fit	0.874	0.882	0.884	0.887	0.884	0.885
	Range	0.027	0.013	0.012	0.010	0.011	0.012
LUNG	Ave.fit	0.709	0.709	0.708	0.709	0.708	0.709
	Std.dev	0.004	0.005	0.005	0.005	0.005	0.004
	Best fit	0.719	0.721	0.718	0.717	0.724	0.715
	Worst fit	0.703	0.702	0.699	0.699	0.699	0.699
	Range	0.017	0.019	0.019	0.018	0.025	0.016
FL CHAIN	Ave.fit	0.757	0.756	0.767	0.765	0.765	0.760
	Std.dev	0.008	0.005	0.005	0.005	0.005	0.005
	Best fit	0.779	0.766	0.777	0.772	0.772	0.769
	Worst fit	0.745	0.743	0.754	0.753	0.752	0.749
	Range	0.033	0.023	0.023	0.019	0.20	0.20
NWTCO	Ave.fit	0.982	0.981	0.980	0.980	0.979	0.983
	Std.dev	0.006	0.005	0.003	0.004	0.005	0.004
	Best fit	0.995	0.987	0.986	0.988	0.986	0.992
	Worst fit	0.969	0.967	0.972	0.971	0.965	0.976
	Range	0.027	0.019	0.014	0.017	0.021	0.016

Table 3: Average initial mutation widths of the ten ensembles with highest validation fitness, where standard deviations are shown in brackets. The average width seem to increase, with some tendencies towards increasing standard deviations, when applying adaptation and self-adaptation to it.

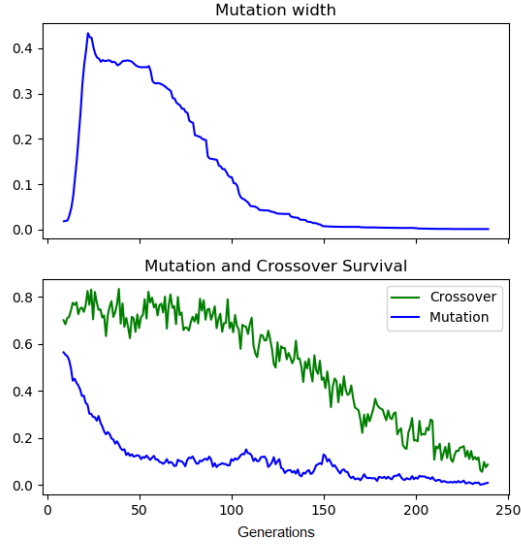
MODEL	FLCHAIN	LUNG	PBC	NWTCO
VANILLA	0.556(0.718)	0.655(0.804)	0.121(0.294)	0.756(1.05)
HEREDITARY	0.57(0.779)	1.02(0.715)	1.07(0.795)	1.1(0.663)
AGA	1.17(0.953)	1.41(0.814)	1.15(0.671)	1.41(0.902)
SAGA-1	1.50(0.592)	0.98(0.876)	1.08(0.913)	1.40(0.829)
SAGA-2	1.03(0.769)	1.40(0.886)	1.46(0.737)	1.07(0.763)
SAGA-3	1.36(0.759)	1.09(0.944)	1.13(0.909)	1.07(0.763)



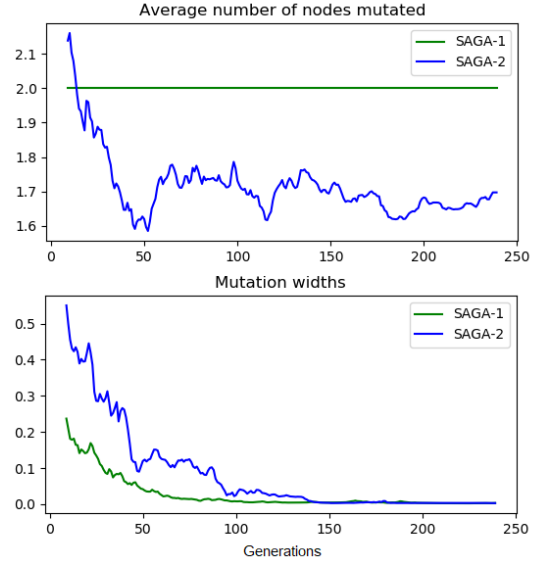
(a) Low mutation rate, $\sigma = 0.1$, test on NWTCO with SAGA-1. Interestingly, the low rate of mutation results in a lower survival rate of crossover offspring when comparing to (b).



(b) High mutation rate, $\sigma = 0.9$, tests on NWTCO with SAGA-1. The evolution of mutation width is similar to the low mutation rate results shown in (a).



(c) Example of quick increase of the mutation width using SAGA-1 when initialising a small mutation width ($\sigma = 0.01$).



(d) Comparison of SAGA-1 and SAGA-2 mutation widths, obtaining the same validation result. Tests of SAGA-2 suggest that the optimal evolution of mutation is to mutate fewer nodes with a larger mutation width.

Figure 1: Evolution of SAGA-1 and SAGA-2

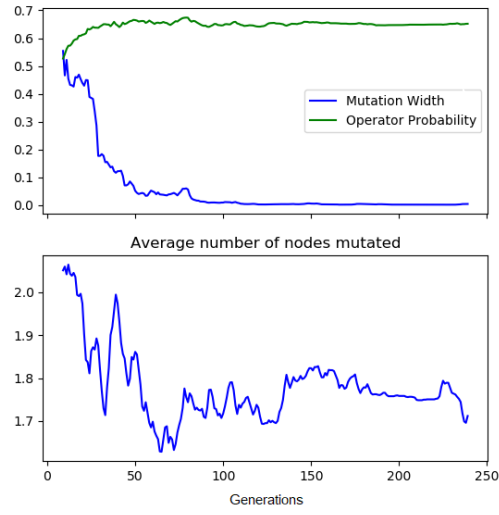


Figure 2: Characteristic self-adaptation of o , m and σ for SAGA-3. Tests of SAGA-3 suggests that the optimal setting for crossover or mutation is close to equal probability of either.

References

- [1] D. J. Montana, L. Davis, *Training Feed-Forward Neural Networks Using Genetic Algorithms*, Proceedings of the 11th International Joint Conference on Artificial Intelligence, San Mateo: 762-767. 1989
- [2] Kalderstam J, Edén P, Bendahl PO, Forsare C, Fernö M, Ohlsson M. *Training artificial neural networks directly on the concordance index for censored data using genetic algorithms*. Artificial Intelligence in Medicine. 2013
- [3] Harrell FE, Lee KL, Mark DB. *Multivariable prognostic models: issues in developing models, evaluating assumptions and adequacy, and measuring and reducing errors* Statistics in Medicine;15(4):361-387. 1996
- [4] Siwe L. *Optimizing genetic training parameters for neural networks in survival analysis*. Lund University. 2018
- [5] Davis L. *Handbook of genetic algorithms*. New York, NY, USA: Van Nostrand Reinhold; 1991
- [6] Meyer-Nieberg S, Beyer HG. *Self-adaptation in evolutionary algorithms*. Berlin Heidelberg: Springer-Verlag; 2007
- [7] Boukhari N, Debbat F, Monmarché N, Slimane M. *A Study on Self-adaptation in the Evolutionary Strategy Algorithm* Oran, Algeria: Springer, Cham ; 2018
- [8] Chellapilla K, Fogel D. *Two new mutation operators for enhanced search and optimization in evolutionary programming*. Villanova University. 1997
- [9] Goldberg DE. *Genetic algorithms in search, optimization and machine learning*. Boston, MA, USA: Addison-Wesley Longman Publishing; 1989
- [10] Box GEP, Muller ME. *A Note on the Generation of Random Normal Deviates*. Princeton University. 1958.
- [11] Smirnov, N. *Table for Estimating the Goodness of Fit of Empirical Distributions*. The Annals of Mathematical Statistics 19, no. 2: 279-81. 1948
- [12] Therneau TM, Grambsch PM. *Modeling Survival Data: Extending the Cox Model*. New York, New York, USA: Springer-Verlag; 2000.
- [13] Loprinzi CL, Laurie JA, Wieand HS, Krook JE, Novotny PJ, Kugler JW, Bartel J, Law M, Bateman M, Klatt NE, et al. *Prospective evaluation of prognostic variables from patient-completed questionnaires. North Central Cancer Treatment Group*. Journal of Clinical Oncology; 12(3): 601-7. 1994.
- [14] Fine JP, Gray RJ. *A proportional hazards model for the subdistribution of a competing risk*. JASA; 94: 496-509. 1999.
- [15] Breslow NE, Chatterjee N. *Design and analysis of two-phase studies with binary outcome applied to Wilms tumour prognosis*. Applied Statistics; 48: 457-468. 1999.
- [16] Andrew YNG. *Feature selection, L_1 vs. L_2 regularization, and rotational invariance*. Stanford University. 2004.
- [17] Hansen N, Ostermeier A. *Completely derandomized self-adaptation in evolution strategies*. Evolutionary Computation: 9(2); 159-195. 2001
- [18] Stanley KO, Miikkulainen R. *Evolving Neural Networks Through Augmenting Topologies*. Evolutionary Computation: 10(2); 99-127. 2002